BUET • CSE 209

Computer Architecture

Tushar Sir's part (ART)

Topic-wise questions with complete worked solutions

68 question cards from the last five term finals — 2019–20 to 2023–24 (CSE 305 and CSE 209) — exactly the questions of CSE209_ART_TF_Practice_Tracker.xlsx. Each question is followed by its solution; numerical answers, ALU designs, assembly programs, encodings and traces were checked by script (all 4-bit ALU inputs simulated, every MIPS program run on a simulator).

Topics follow the course slides (Chapter 1, ALU Design, Chapter 2, Chapter 3). Questions on material outside the short slides are kept in a separate, flagged topic at the end. Inside each topic the most recent paper comes first. Page numbers ("TF page") refer to CSE209 ALL TF QUES (TF209).pdf. Two questions are shared with the RS part (2022–23 Q11(b) and Q12(c)); only the ART halves are solved here.

Credit: Claude

Computer Abstractions and Performance

Chapter 1 · Lessons L1–L4 · 10 questions

2023–24 • Q6(a)

CSE 209 • TF page 6 • MARKS 13

Our favourite program runs in 10 s on computer A (4 GHz clock). The designer wants computer B to run it in 6 s; a substantial clock-rate increase is possible but makes B need 1.2 times as many clock cycles as A. What clock rate should we tell the designer to target?

SOLUTION

Cycles_A = CPU time × clock rate = 10 s × 4×10^9 = 40×10^9.

Cycles_B = 1.2 × 40×10^9 = 48×10^9.

Clock rate_R = Cycles_R / CPU time_R = 48×10^9 / 6 s = 8×10^9 Hz.

Answer. Target clock rate of B = **8 GHz**.

2022–23 • Q6(d)

CSE 305 • TF page 19 • MARKS 15

P1 (2.7 GHz, CPI of classes A,B,C,D = 2,1,3,2) and P2 (3.2 GHz, CPI 2,3,2,3) implement the same ISA. A program of 1.2×10^6 instructions has 20 % class A, 25 % B, 40 % C, 15 % D. (i) Clock cycles on each. (ii) Which is faster and by how much? (iii) If 40 % of the code is improved, what is the maximum possible improvement factor?

SOLUTION

Instructions: A 240 000, B 300 000, C 480 000, D 180 000.

```
P1 cycles = 240k×2 + 300k×1 + 480k×3 + 180k×2 = 2.58×10^6   T1 = 2.58e6 / 2.7e9 = 0.956 ms
P2 cycles = 240k×2 + 300k×3 + 480k×2 + 180k×3 = 2.88×10^6   T2 = 2.88e6 / 3.2e9 = 0.900 ms
```

Answer. (i) 2.58×10^6 and 2.88×10^6 cycles. (ii) **P2 is faster** by 0.956/0.900 = 1.06×.

(iii) Amdahl with an infinitely fast improved part: S_{max} = 1/(1 − 0.4) = **1.67**.

2022–23 • Q7(a)

CSE 305 • TF page 19 • MARKS 5

Define response time and throughput. Are these two terms synonymous?

SOLUTION

Response time (execution time, latency) is the time to complete one task, from start to finish.
Throughput is the total work done per unit time (tasks or transactions per hour).

Answer. Not synonymous. A faster processor lowers response time and so also raises throughput; extra processors raise throughput (more tasks at once) but leave the response time of a single task unchanged.

2021–22 • Q5(b)

CSE 305 • TF page 23–24 • MARKS 10

2 % of a program's runtime is not parallelizable. It runs on 24 cores of the same speed with no extra overhead. (i) What is the parallel speedup for 24 cores? (ii) Is it possible to make the program 60 times faster? If yes, how; if not, why?

SOLUTION

Amdahl: Speedup = 1 / ((1 − f) + f/p), f = 0.98, p = 24.

```
S = 1 / (0.02 + 0.98/24) = 1 / (0.02 + 0.04083) = 16.44
```

Answer. (i) **16.44**.

(ii) The limit is 1/0.02 = 50 < 60, so **no** — not by adding cores. It becomes possible only by making the serial part smaller (< 1/60 ≈ 1.67 % of the time, by redesigning the algorithm) or by running the serial part on a faster core.

2021–22 • Q7(b)

CSE 305 • TF page 25 • MARKS 16

Your research partner says FLOPS is a misleading metric for the general performance of a processor but could still be useful for a specific part or type of processor. Do you agree? Explain in detail.

SOLUTION

Agree. FLOPS counts only floating-point operations per second:

• Integer, memory and branch work is not measured at all, so programs dominated by it look slow although they are not.

• Different ISAs have different FP operations (fused multiply-add counts as one or two), so FLOPS of different machines are not comparable.

• Like MIPS, it varies from program to program on one machine; execution time of real programs is the only general measure.

Answer. For a **specific part** — the FP unit, a GPU, a vector unit — or for FP-dominated work (matrix multiply, simulation, deep learning) FLOPS directly measures how fast that part does its job, so it is a useful specialised metric.

2021–22 • Q8(a)

CSE 305 • TF page 25 • MARKS 11

200 000 instructions: 25 % class A, 60 % B, 15 % C. P1 2.7 GHz (CPI 3,3,2), P2 3.5 GHz (CPI 2,4,1). (i) Total clock cycles on both. (ii) Execution times; which is faster? (iii) By how much must the CPI of class C improve so the program runs 2× faster on the better implementation?

SOLUTION

A = 50 000, B = 120 000, C = 30 000 instructions.

```
P1 = 50k×3 + 120k×3 + 30k×2 = 570 000 cycles → T = 570 000 / 2.7 GHz = 211.1 µs
P2 = 50k×2 + 120k×4 + 30k×1 = 610 000 cycles → T = 610 000 / 3.5 GHz = 174.3 µs
```

Answer. P2 is faster (211.1/174.3 = 1.21×).

(iii) 2× in P2 → 305 000 cycles. Classes A and B alone need 100 000 + 480 000 = 580 000 > 305 000, so even $CPI_C = 0$ is not enough: **impossible**.

2020–21 • Q10(a)

CSE 305 • TF page 30 • MARKS 12

Briefly explain the eight great ideas in computer architecture.

SOLUTION

Idea	Meaning	Example
Design for Moore's law	design for the transistor budget available when the product ships	plan for 2× transistors in 2 years
Use abstraction	hide lower-level detail behind a simpler model	ISA hides implementation
Make the common case fast	speed up the frequent case	immediates, PC-relative branches
Performance via parallelism	do several operations at once	multicore, 31 adders in a multiplier
Performance via pipelining	overlap steps of successive operations	5-stage MIPS pipeline
Performance via prediction	guess, start, recover if wrong	branch prediction, prefetching
Hierarchy of memories	fast/small/expensive on top, slow/large/cheap below	registers → cache → DRAM → disk
Dependability via redundancy	spare components take over on failure	RAID, ECC

2020–21 • Q12(a)

CSE 305 • TF page 33 • MARKS 8

Instruction types with CPI: multiplication 3, addition 1, subtraction 2, division 4. Program A has 2, 1, 4, 2 instructions; program B has 3, 2, 3, 2. Which program has the higher average clock cycles per instruction?

SOLUTION

CPI = Σ $IC_i \cdot CPI_i$ / Σ IC_i.

```
A: cycles = 2×3 + 1×1 + 4×2 + 2×4 = 23, IC = 9  → CPI = 2.56
B: cycles = 3×3 + 2×1 + 3×2 + 2×4 = 25, IC = 10 → CPI = 2.50
```

Answer. Program A has the higher average CPI (2.56 vs 2.50); B still needs more total cycles because it executes more instructions.

2019–20 • Q1(a)

CSE 305 • TF page 36 • MARKS 15

Program 1 is run on systems A (2 GHz) and B (2.5 GHz). Logs of A: Mul 4, Mul 4, Div 5, Div 5, Add 2 cycles for Program 1 (other entries belong to Programs 2–3 or are OS I/O overhead); logs of B: Mul 5, Mul 5, Div 6, Div 6, Add 3 for Program 1. Compare their performance on the execution time of Program 1.

SOLUTION

Only Program 1's own operations count (CPU time); the other programs' lines and the OS I/O overhead are ignored.

```
A: cycles = 4+4+5+5+2 = 20 → T_A = 20 / 2 GHz   = 10 ns
B: cycles = 5+5+6+6+3 = 25 → T_B = 25 / 2.5 GHz = 10 ns
```

Answer. $Performance_A / Performance_B = T_B / T_A = 1$ — the two systems perform **equally** on Program 1. B's faster clock is cancelled by its extra cycles.

2019–20 • Q1(b)

CSE 305 • TF page 36 • MARKS 12

What is the importance of Moore's law? "To increase the overall performance of a computer by x, identify the most important aspect and improve its performance by x." Agree or disagree? Justify with mathematics.

SOLUTION

Moore's law: transistor count doubles about every 18–24 months. It makes hardware cheaper and faster and is why designers must design for the technology available when the product ships.

Disagree (Amdahl's law). If the improved aspect takes fraction f < 1 of the time:

```
Speedup = 1 / ((1 − f) + f/x)  < x   for every x > 1
```

Example: f = 0.8, x = 4 → 1/(0.2 + 0.2) = 2.5, not 4. Even $x \to \infty$ gives only 1/(1 − f) = 5. Overall speedup x is reached only when f = 1.

Answer. The unimproved part (1 − f) sets a ceiling; improve the **common case** (largest f) but expect less than x.

Arithmetic Logic Unit (ALU) Design

ALU Design · Lessons L6–L9 · 8 questions

2023–24 • Q5(b)

CSE 209 • TF page 5 • MARKS 15

Design a 4-bit ALU (A, B; selection S2 S1 S0) with parallel adders: 000 F = A − B − 1; 001 F = A + B + 1; 010 transfer A; 011 decrement A; 10x F = A OR B; 11x F = A'. Devise the input equations X_i, Y_i, Z_i.

SOLUTION

S2 S1 S0	Function	X_i	Y_i	C_{in}
000	A − B − 1 = A + B'	A	B'	0
001	A + B + 1	A	B	1
010	transfer A = A + 1s + 1	A	1	1
011	A − 1 = A + 1s	A	1	0
10x	A OR B	A + B	0	(carries 0)
11x	A'	A	1	(carries 0)

In logic mode (S2 = 1) all carries are forced to 0, so $F_i = X_i \oplus Y_i$.

```
X_i = A_i + S2·S1'·B_i
Y_i = S1 + S2'·(B_i ⊙ S0)          (B_i ⊙ S0 = B_i' for S0 = 0, B_i for S0 = 1)
C_in = S2'·(S1 ⊕ S0)               Z_i = S2'·C_i
```

Answer. Check: 011 → Y = 1, C_{in} = 0 → A + 1111 = A − 1 ✓; 11x → X = A, Y = 1 → A ⊕ 1 = A' ✓. (All six functions verified by simulation for every A, B.)

2022–23 • Q11(a)

CSE 209 • TF page 13 • MARKS 7

Why is the overflow bit (V) computed by XOR-ing the last two carry bits in binary arithmetic operations?

SOLUTION

Let C_n be the carry into the sign position and C_{n+1} the carry out of it.

Operand signs	C_{n+1}	Sign of result	Overflow when
+ and + (MSBs 0,0)	always 0	= C_n	C_n = 1 (result looks negative)
− and − (MSBs 1,1)	always 1	= C_n	C_n = 0 (result looks positive)
+ and −	= C_n	—	never

In both overflow cases the two carries differ; when they are equal there is no overflow.

Answer. $V = C_n \oplus C_{n+1}$ ($C_8 \oplus C_9$ for 8 bits). Examples (4 bits): 0111 + 0001 = 1000 — carry in 1, out 0 → V = 1; 1000 + 1111 = (1)0111 — in 0, out 1 → V = 1.

2022–23 • Q11(d)

CSE 209 • TF page 13 • MARKS 10

Design a 4-bit ALU, selection S2 S1 S0: 000 −A − B − 2; 001 add 1's complement of A to B; 010 A + B; 011 A − B; 10x A AND B; 11x A XOR B. Derive X_i, Y_i, Z_i.

SOLUTION

S2 S1 S0	Function	X_i	Y_i	C_{in}
000	−A − B − 2 = A' + B'	A'	B'	0
001	B + A'	A'	B	0
010	A + B	A	B	0
011	A − B = A + B' + 1	A	B'	1
10x	A AND B	A + B'	B'	—
11x	A XOR B	A	B	—

AND from the adder with carries 0: (A + B') ⊕ B' = AB (B = 0 → 1⊕1 = 0; B = 1 → A⊕0 = A).

```
X_i = (A_i ⊕ S2'S1') + S2·S1'·B_i'
Y_i = B_i ⊙ (S1 ⊕ S2'S0)
C_in = S2'·S1·S0                  Z_i = S2'·C_i
```

Answer. Verified by simulation for all 4-bit A, B and every code.

2022–23 • Q5(b)

CSE 305 • TF page 18 • MARKS 14

Design a 4-bit ALU using only parallel SUBTRACTORS and logic gates; S2 S1 S0: 000 A − B; 001 A + B; 010 −A − B − 2; 011 B − A; 100 A XOR B; 110 A AND B.

SOLUTION

A full-subtractor stage gives D = X ⊕ Y ⊕ Z, borrow = X'Y + X'Z + YZ: the unit computes **F = X − Y − B_{in}**. Use −B' = B + 1.

S2 S1 S0	F	as X − Y − B_{in}	X_i	Y_i	B_{in}
000	A − B	A − B − 0	A	B	0
001	A + B	A − B' − 1	A	B'	1
010	−A − B − 2	A' − B − 1	A'	B	1
011	B − A	A' − B'	A'	B'	0
100	A XOR B	borrows 0	A	B	—
110	A AND B	borrows 0	A + B'	B'	—

```
X_i = (A_i ⊕ S2'S1) + S2·S1·B_i'
Y_i = B_i ⊕ (S0 + S2·S1)
B_in = S2'·(S1 ⊕ S0)              borrow into stage i: Z_i = S2'·B_i
```

Answer. Codes 101 and 111 are unused (don't-cares). Verified by simulation.

2020–21 • Q10(b)

CSE 305 • TF page 30–31 • MARKS 18

Design a 4-bit ALU with selection S3 S2 S1 S0: 0000 A; 0001 A−1; 0010 A+B+1; 0011 A+B; 0100 A−B; 0101 A−B−1; 0110 A+1; 0111 A; 100x A'; 101x AB'; 110x A ⊙ B; 111x A ∨ B. Derive X_i, Y_i, Z_i with the least number of gates.

SOLUTION

S3 S2 S1 S0	Function	X_i	Y_i	C_{in}
0000	A	A	0	0
0001	A − 1	A	1	0
0010	A + B + 1	A	B	1
0011	A + B	A	B	0
0100	A − B	A	B'	1
0101	A − B − 1	A	B'	0
0110	A + 1	A	0	1
0111	A	A	0	0
100x	A'	A	1	—
101x	AB'	A + B	B	—
110x	A ⊙ B	A	B'	—
111x	A ∨ B	A + B	0	—

```
X_i = A_i + S3·S1·B_i
Y_i = S2'S1'(S0 + S3) + S2'S1·B_i + S2S1'·B_i'
C_in = S3'·S0'·(S2 + S1)          Z_i = S3'·C_i
```

Answer. Verified by simulation for all 4-bit inputs and all 16 codes.

2020–21 • Q12(d)

CSE 305 • TF page 34 • MARKS 8

A module takes two 4-bit inputs and shows only V, Z, S, C of the subtraction (second input from the first). If the first input is A and the second B, draw the complete circuit that determines whether A > B, A = B or A < B.

SOLUTION

Feed A (first) and B (second); the module computes A − B = A + B' + 1 (signed 2's complement).

```
A = B  :  Z
A < B  :  S ⊕ V          (S alone is wrong if the subtraction overflows)
A > B  :  (S ⊕ V)' · Z'   = NOR(S ⊕ V, Z)
```

Gates: one XOR (S, V), the equality output is Z itself, and one NOR (S⊕V, Z) for A > B.

Answer. For unsigned inputs use A < B = C', A = B = Z, A > B = C·Z'.

2019–20 • Q2(a)

CSE 305 • TF page 36–37 • MARKS 20

4-bit ALU, S3 S2 S1 S0: 0000 A'; 0001 A'+1; 0010 A'+B; 0011 A'+B+1; 0100 A'−B−1; 0101 A'−B; 0110 A'−1; 0111 A'; 100x (AB)'; 101x A ⊙ B; 110x A ⊕ B; 111x A ∨ B. Derive X_i, Y_i, Z_i using the least gates.

SOLUTION

This is the standard arithmetic table with A' in place of A. Logic rows are chosen so that Y keeps the pattern 0, B, B', 1.

S3 S2 S1 S0	Function	X_i	Y_i	C_{in}
000x / 001x...	A', A'+B, A'−B−1, A'−1 (+1 for odd S0)	A'	0, B, B', 1	S0
100x	(AB)' NAND	A' + B'	0	—
101x	A ⊙ B XNOR	A'	B	—
110x	A ⊕ B XOR	A'	B'	—
111x	A ∨ B OR	A'B'	1	—

```
Y_i = S1·B_i + S2·B_i'
X_i = A_i'·(S3·S2·S1·B_i)' + S3·S2'·S1'·B_i'
C_in = S3'·S0                    Z_i = S3'·C_i
```

Answer. Check NAND (100x): X = A' + B', Y = 0 → A' + B' = (AB)' ✓; OR (111x): X = A'B', Y = 1 → (A'B')' = A + B ✓. Verified by simulation.

2019–20 • Q2(b)

CSE 305 • TF page 37 • MARKS 15

Implement the hardware for F = 1 if ((A ≥ B && C ≠ D) || X == 0) else 0, with 4-bit A, B, C, D, X, basic gates and status-generator modules (each shows V, Z, S, C of 2nd input subtracted from 1st). Just draw the circuit.

SOLUTION

Three modules: M1(A, B), M2(C, D), M3(X, 0000).

```
A >= B  = (S1 ⊕ V1)'        from M1  (signed; unsigned: C1)
C != D  = Z2'                from M2
X == 0  = Z3                 from M3
F = (S1 ⊙ V1) · Z2' + Z3
```

Answer. Gates: one XNOR (S1, V1), one NOT (Z2), one AND, one OR plus the three modules.

Instructions: Language of the Computer

Chapter 2 · Lessons L10–L18 · 30 questions

2023–24 • Q5(a)(ii)

CSE 209 • TF page 5 • MARKS 10

MIPS-v22: R-type = opcode(6) | rt(5) | rd(5) | rs(5) | shamt(5) | funct(6); I-type = opcode(6) | rt(5) | rs(5) | constant(16). $t0, $t1, $t2 = 2, 3, 4; shamt = funct = 0. Translate to binary: add $t0,$t1,$t2 (opcode 0; $t0 destination, $t1 source 1, $t2 source 2); addi $t2,$t0,1 (opcode 6); lw $t1,4($t0) (opcode 5).

SOLUTION

The field order is the question's, not standard MIPS. In I-type rt is the destination (addi, lw).

```
add  $t0,$t1,$t2: op 0 | rt=$t2=4 | rd=$t0=2 | rs=$t1=3 | 0 | 0
   000000 00100 00010 00011 00000 000000 = 0x00821800
addi $t2,$t0,1:   op 6 | rt=$t2=4 | rs=$t0=2 | 1
   000110 00100 00010 0000000000000001 = 0x18820001
lw   $t1,4($t0):  op 5 | rt=$t1=3 | rs=$t0=2 | 4
   000101 00011 00010 0000000000000100 = 0x14620004
```

Answer. Machine words 0x00821800, 0x18820001, 0x14620004.

2023–24 • Q6(b)

CSE 209 • TF page 6 • MARKS 13

MIPS for: int a=5, b=10, c=2; int N=30; do { if ((a<<2) > b) { a += 2; b += a; } else { a += c; b += c; } } while (a < N); return 0; ($t0=a, $t1=b, $t2=c, $t3=N)

SOLUTION

(a << 2) > b ⇔ b < (a << 2), so slt with the operands swapped. Test the opposite condition to skip the then-part.

```
main:  addi $t0, $zero, 5      # a = 5
       addi $t1, $zero, 10     # b = 10
       addi $t2, $zero, 2      # c = 2
       addi $t3, $zero, 30     # N = 30
Loop:  sll  $t4, $t0, 2        # t4 = a << 2
       slt  $t5, $t1, $t4      # t5 = 1 if b < (a<<2), i.e. (a<<2) > b
       beq  $t5, $zero, Else   # condition false -> else part
       addi $t0, $t0, 2        # a += 2
       add  $t1, $t1, $t0      # b += a
       j    Cond
Else:  add  $t0, $t0, $t2      # a += c
       add  $t1, $t1, $t2      # b += c
Cond:  slt  $t5, $t0, $t3      # t5 = 1 if a < N
       bne  $t5, $zero, Loop   # repeat while a < N
       add  $v0, $zero, $zero  # return 0
       jr   $ra
```

Answer. Simulated: ends with a = 31, b = 121, like the C program.

2023–24 • Q6(c)

CSE 209 • TF page 6 • MARKS 9

Explain PC-relative addressing mode, immediate addressing mode and base addressing mode with examples.

SOLUTION

PC-relative — the 16-bit field of beq/bne is a word offset from PC + 4: target = PC + 4 + offset×4. Example: beq at 80012 with offset 2 → 80016 + 8 = 80024.

Immediate — the operand is the 16-bit constant in the instruction (sign-extended): addi $s3,$s3,4.

Base (displacement) — memory address = register + sign-extended 16-bit constant: lw $t0,32($s3) with $s3 = 1000 → address 1032.

2023–24 • Q8(c)

CSE 209 • TF page 7 • MARKS 5

beq $s0,$s1,L1 where L1 is out of branching range. Replace the beq by a pair of instructions that offers a much greater branching distance.

SOLUTION

The 16-bit branch offset reaches only ±32 K words. Invert the condition and use a jump (26-bit field, reaches the whole 256 MB block):

```
bne $s0, $s1, L2     # skip the jump when NOT equal
j   L1               # far target
L2: ...
```

Answer. Same behaviour: control reaches L1 exactly when $s0 = $s1.

2022–23 • Q11(b) NOT part

CSE 209 • TF page 13

How is the NOT operation performed in MIPS assembly? (The jal-versus-j part of 11(b) is in the RS tracker.)

SOLUTION

MIPS has no NOT instruction; it has the three-operand NOR (a NOR b = NOT(a OR b)).

```
nor $t0, $t1, $zero     # $t0 = ~($t1 | 0) = ~$t1
```

Answer. NOR with $zero gives NOT, so no separate instruction is needed (simplicity favours regularity).

2022–23 • Q11(c)

CSE 209 • TF page 13 • MARKS 10

What are the different addressing modes of MIPS? Illustrate the calculation process for each.

SOLUTION

Mode	Operand / target	Calculation	Example
1. Immediate	constant in the instruction	sign-extend(imm16)	addi $s3,$s3,4
2. Register	a register	R[rs], R[rt]	add $t0,$s1,$s2
3. Base (displacement)	memory word	address = R[rs] + sign-extend(imm16)	lw $t0,32($s3)
4. PC-relative	branch target	PC + 4 + sign-extend(imm16) × 4	beq $s0,$s1,L1
5. Pseudo-direct	jump target	(PC+4)[31:28] : addr26 : 00	j Loop

```
Immediate    | op | rs | rt |  imm16  |  operand = imm16
Register     | op | rs | rt | rd | .. | funct |  operand = R[rs], R[rt]
Base         | op | rs | rt |  addr16 |  R[rs] + addr16 → memory
PC-relative  | op | rs | rt |  addr16 |  PC+4 + addr16×4 → memory (instruction)
Pseudodirect | op |      addr26       |  PC+4[31:28] : addr26 : 00
```

Numbers: lw $t0,32($s3), $s3 = 1000 → 1032. beq at 80012, offset 2 → 80024. j 20000 at 80020 → 0000 : 20000×4 = 80000.

2022–23 • Q12(a)

CSE 209 • TF page 13 • MARKS 5

How does Just-In-Time (JIT) compilation improve the performance of a Java application?

SOLUTION

Java is compiled to portable bytecodes that the JVM interprets — portable but slow. The JIT compiler finds the **hot** (frequently executed) methods while the program runs and compiles them once to native code for the host machine; later calls run at native speed.

Answer. The program keeps Java's portability but approaches compiled-code performance; the compile cost is paid only for hot code.

2022–23 • Q12(b)

CSE 209 • TF page 13 • MARKS 10

What is the main difference between static linking and dynamic linking? What are the advantages of using dynamic linking?

SOLUTION

Static: all library routines that may be called are copied into the executable at link time. **Dynamic**: a library routine is linked and loaded only when it is first called.

Advantages of dynamic linking: (1) avoids image bloat from statically linking every (transitively) referenced library; (2) automatically picks up new library versions; (3) one library copy is shared in memory by many programs.

Answer. Dynamic linking uses lazy linkage — the first call goes through a stub to the dynamic linker, which maps the routine and patches the indirection so later calls jump directly.

2022–23 • Q12(c)(i),(iii)

CSE 209 • TF page 14 • MARKS 7

(i) Write MIPS that creates the 32-bit constant 0010 0000 0000 0001 0100 1001 0010 0100_2 in $t1. (iii) If PC = 0x1FFFF000, can a single branch instruction reach the address equal to that constant? (12(c)(ii) is in the RS tracker.)

SOLUTION

The constant is 0x2001 4924.

```
lui $t1, 0x2001          # 8193: upper 16 bits
ori $t1, $t1, 0x4924     # 18724: lower 16 bits
```

(iii) PC + 4 = 0x1FFFF004. Offset = (0x20014924 − 0x1FFFF004)/4 = 0x5648 = 22 088 words ≤ 32 767.

Answer. Yes, a branch can reach it (the target is word-aligned and 22 088 words away, inside ±32 K).

2022–23 • Q9(c)

CSE 209 • TF page 12 • MARKS 10

MIPS for: int fun(int a, int b) { if (b == 0) return 0; else if (b / 2 == 0) return fun(a*4, b/8); return fun(a+a, b/4) + a; }

SOLUTION

a in $a0, b in $a1. b/2, b/4, b/8 by sra (b ≥ 0 assumed). The value a must survive the second call, so it is saved on the stack.

```
fun:   bne  $a1, $zero, L1     # b != 0 -> recursive cases
       add  $v0, $zero, $zero  # return 0
       jr   $ra
L1:    addi $sp, $sp, -8       # frame for $ra and a
       sw   $ra, 4($sp)
       sw   $a0, 0($sp)        # save a (needed after the 2nd call)
       sra  $t0, $a1, 1        # t0 = b / 2   (b >= 0 assumed)
       bne  $t0, $zero, L2
       sll  $a0, $a0, 2        # arg1 = a * 4
       sra  $a1, $a1, 3        # arg2 = b / 8
       jal  fun                # v0 = fun(a*4, b/8)
       j    Done
L2:    add  $a0, $a0, $a0      # arg1 = a + a
       sra  $a1, $a1, 2        # arg2 = b / 4
       jal  fun                # v0 = fun(a+a, b/4)
       lw   $t1, 0($sp)        # t1 = original a
       add  $v0, $v0, $t1      # v0 = fun(...) + a
Done:  lw   $ra, 4($sp)
       addi $sp, $sp, 8
       jr   $ra
```

Answer. Simulated against the C function for 6 argument pairs — all equal.

2022–23 • Q5(a)

CSE 305 • TF page 18 • MARKS 7

Rachel adds lh, lhu, sh and shu for multi-language support. Ross argues she does not need all four. Do you agree with Ross?

SOLUTION

Stores write only the rightmost 16 bits, so there is nothing to sign- or zero-extend: shu would do exactly what sh does.

For loads the extension matters: lhu (zero-extend) is what characters need; lh (sign-extend) is for signed 16-bit integers.

Answer. Agree with Ross — lh, lhu and sh are enough; shu is redundant.

2022–23 • Q5(c)

CSE 305 • TF page 18, 20 • MARKS 14

Assemble the object files of procedures A (text 0x150, data 0x40: lw $a0,0($gp) needs X; jal 0 needs B) and B (text 0x350, data 0x50: sw $a1,0($gp) needs Y; jal 0 needs A). Find the updated addresses of the first few instructions of the executable. Memory map: text at 0x00400000, static data at 0x10000000, $gp = 0x10008000.

SOLUTION

Executable: text size 0x150 + 0x350 = 0x4A0, data size 0x40 + 0x50 = 0x90. A's text starts at 0x00400000, B's at 0x00400150; X = 0x10000000, Y = 0x10000040.

Address	Instruction	Patched value
0x00400000	lw $a0, 0x8000($gp)	X − $gp = 0x10000000 − 0x10008000 = −0x8000
0x00400004	jal 0x00400150	field = 0x00400150 / 4 = 0x100054
0x00400150	sw $a1, 0x8040($gp)	Y − $gp = 0x10000040 − 0x10008000 = −0x7FC0 = 0x8040
0x00400154	jal 0x00400000	field = 0x100000
0x10000000 / 0x10000040	(X) / (Y)	A's data / B's data

Answer. Linker steps: merge segments, resolve labels (X, Y, A, B), patch location-dependent references.

2022–23 • Q6(b)

CSE 305 • TF page 18 • MARKS 7

What support does the MIPS instruction-set architecture provide for synchronization? Explain with an example.

SOLUTION

Two processors sharing memory need an **atomic** read-modify-write, otherwise a data race occurs (result depends on access order). MIPS provides **load linked** `ll rt, off(rs)` and **store conditional** `sc rt, off(rs)`: sc stores only if the location was not changed since the ll and returns 1 in rt (success) or 0 (failure).

```
try: add $t0, $zero, $s4   # copy exchange value
     ll  $t1, 0($s1)       # load linked
     sc  $t0, 0($s1)       # store conditional
     beq $t0, $zero, try   # retry if the store failed
     add $s4, $zero, $t1   # old value into $s4
```

Answer. This is an atomic swap — usable to acquire a lock.

2022–23 • Q6(c)

CSE 305 • TF page 19 • MARKS 8

MIPS for: if (x > y + 131075) x = 8; with x in $t1 and y in $t2.

SOLUTION

131075 = 0x00020003 does not fit a 16-bit immediate, so build it with lui/ori.

```
frag:  lui  $t0, 2             # t0 = 131072
       ori  $t0, $t0, 3        # t0 = 131075
       add  $t0, $t2, $t0      # t0 = y + 131075
       slt  $t3, $t0, $t1      # t3 = 1 if y + 131075 < x
       beq  $t3, $zero, Skip
       addi $t1, $zero, 8      # x = 8
Skip:  jr   $ra
```

Answer. Simulated for four (x, y) pairs — correct.

2022–23 • Q7(b)

CSE 305 • TF page 19 • MARKS 7

Chandler's program uses 40 variables but MIPS has only 32 registers. What does the compiler do?

SOLUTION

It performs register allocation: the most frequently used variables stay in registers and the rest live in memory (the stack frame). When one is needed it is loaded with lw and written back with sw — **spilling registers**.

Answer. The compiler spills the least frequently used variables, because each spill costs extra instructions and slower memory accesses.

2022–23 • Q7(c)

CSE 305 • TF page 19 • MARKS 8

Translate int f(int a, int b, int c, int d) { return func(func(a, b), c + d); } into MIPS (use lower-numbered $t registers first). func is int func(int a, int b).

SOLUTION

c + d must be saved before the first call because $t registers are not preserved by the callee.

```
f:     addi $sp, $sp, -8       # frame: $ra and c+d
       sw   $ra, 4($sp)
       add  $t0, $a2, $a3      # t0 = c + d
       sw   $t0, 0($sp)        # $t0 is not preserved across a call
       jal  func               # v0 = func(a, b)   (a, b already in $a0, $a1)
       add  $a0, $v0, $zero    # arg1 = func(a, b)
       lw   $a1, 0($sp)        # arg2 = c + d
       jal  func               # v0 = func(func(a,b), c+d)
       lw   $ra, 4($sp)
       addi $sp, $sp, 8
       jr   $ra
```

Answer. Simulated with a test func — result matches func(func(a,b), c+d).

2022–23 • Q7(d)

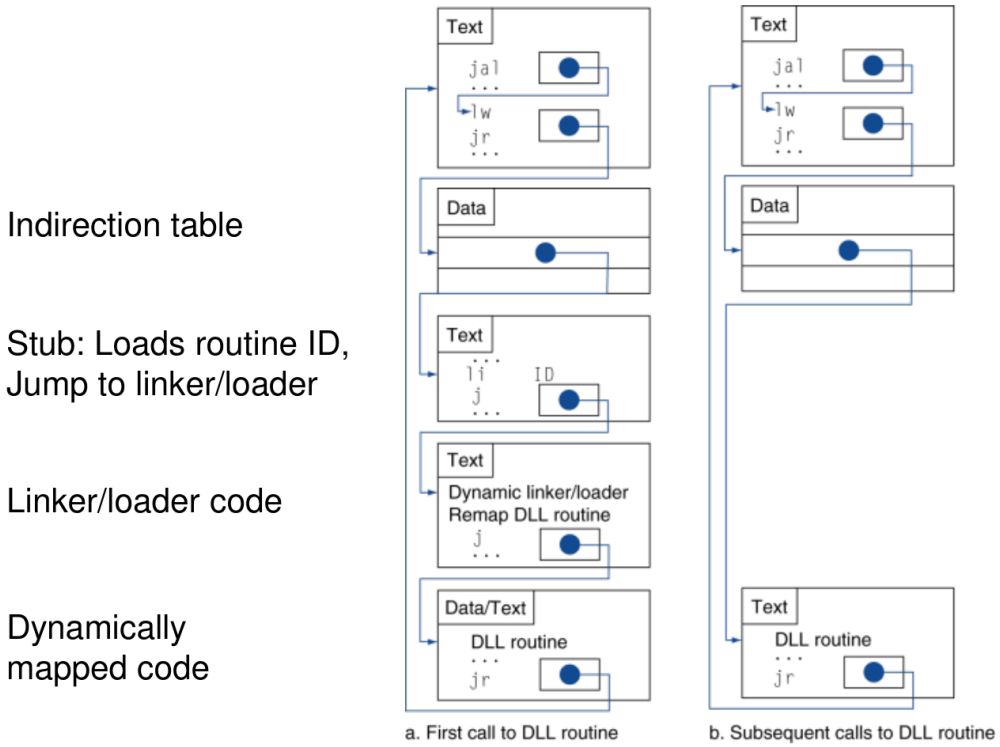
CSE 305 • TF page 19 • MARKS 15

Write about the main difference between static linking and dynamic linking. With the necessary diagrams, explain how dynamically linked libraries (DLLs) are called.

SOLUTION

Static vs dynamic: see 2022-23 12(b). **Lazy linkage** (how a DLL is called):

1. The first call to a library routine jumps through an **indirection table** to a **stub**.
2. The stub loads the routine ID and jumps to the **dynamic linker/loader**.
3. The linker finds the routine, maps it into memory (the dynamically mapped code) and changes the indirect address.
4. Later calls go through the table straight to the DLL routine — no linker involved.



Lazy linkage: (a) first call via stub and linker; (b) subsequent calls jump directly (Ch. 2, sl. 64)

2021-22 • Q5(a)

CSE 305 • TF page 23 • MARKS 15

Which design principle (simplicity favours regularity; smaller is faster; good design demands good compromise; make the common case faster) is behind each: (i) keeping a conditional branch's offset in 16 bits; (ii) keeping 32 registers rather than many more; (iii) consistent instruction format — always three registers for R-type; (iv) multiple instruction formats for flexibility; (v) reduced instruction set, complex instructions (pseudo bgt, bge, move) translated into basic ones.

SOLUTION

	Principle	Why
(i)	Good design demands good compromises (also common case)	a 16-bit offset keeps the instruction 32 bits; most targets are near
(ii)	Smaller is faster	a larger register file has longer wires and slower access; 5-bit fields fit 32 bits
(iii)	Simplicity favours regularity	same hardware path for every arithmetic instruction
(iv)	Good design demands good compromises	three formats keep every instruction 32 bits; op, rs, rt stay aligned
(v)	Simplicity favours regularity (and common case fast)	few simple instructions; the assembler expands pseudoinstructions via $at

2021-22 • Q6(a)

CSE 305 • TF page 24 • MARKS 15

MIPS assembly for: int fib(int n) { if (n==0) return 0; else if (n==1) return 1; else return fib(n−1) + fib(n−2); }

SOLUTION

slti n < 2 handles both base cases (return n). Two recursive calls: fib(n−1) is kept on the stack across the second call.

```
fib:   slti $t0, $a0, 2        # n < 2 ?  (n = 0 -> 0, n = 1 -> 1)
       beq  $t0, $zero, Rec
       add  $v0, $a0, $zero    # return n
       jr   $ra
Rec:   addi $sp, $sp, -12      # room for $ra, n, fib(n-1)
       sw   $ra, 8($sp)
       sw   $a0, 4($sp)
       addi $a0, $a0, -1
       jal  fib                # v0 = fib(n-1)
       sw   $v0, 0($sp)        # keep it across the next call
       lw   $a0, 4($sp)
       addi $a0, $a0, -2
       jal  fib                # v0 = fib(n-2)
       lw   $t1, 0($sp)
       add  $v0, $v0, $t1      # fib(n-1) + fib(n-2)
       lw   $ra, 8($sp)
       addi $sp, $sp, 12
       jr   $ra
```

Answer. Simulated: fib(0..12) = 0, 1, 1, 2, 3, 5, 8, 13, ... 144.

2021–22 • Q6(b)(ii)

CSE 305 • TF page 24 • MARKS 5

PC = 0x2000 0000. Is it possible to use a single branch (beq/bne) instruction to set the PC to 0x4000 0000? ((i) about j is in the RS tracker.)

SOLUTION

Branch target = PC + 4 + offset × 4 with a 16-bit signed offset: reach ±32 K words (±128 KB).

Distance = (0x40000000 − 0x20000004)/4 = 0x07FFFFFF ≈ 134 million words ≫ 32 767.

Answer. No. Use lui $t0, 0x4000 and jr $t0 (a j also fails: top four bits 0010 ≠ 0100).

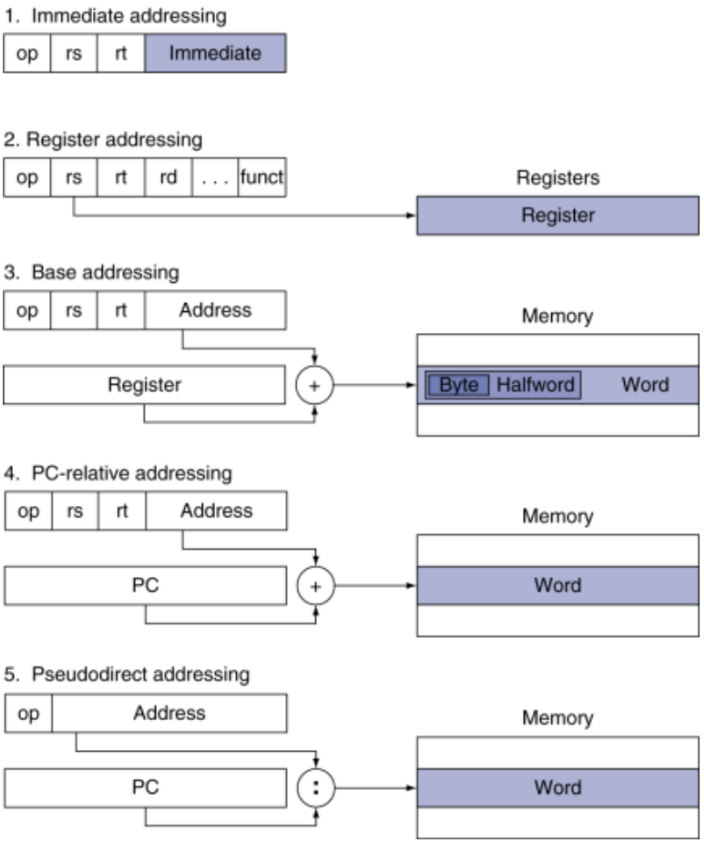
2021–22 • Q8(c)

CSE 305 • TF page 25 • MARKS 10

What are the different addressing modes of MIPS? Illustrate the calculation process for each addressing mode.

SOLUTION

Mode	Operand / target	Calculation	Example
1. Immediate	constant in the instruction	sign-extend(imm16)	addi $s3,$s3,4
2. Register	a register	R[rs], R[rt]	add $t0,$s1,$s2
3. Base (displacement)	memory word	address = R[rs] + sign-extend(imm16)	lw $t0,32($s3)
4. PC-relative	branch target	PC + 4 + sign-extend(imm16) × 4	beq $s0,$s1,L1
5. Pseudo-direct	jump target	(PC+4)[31:28] : addr26 : 00	j Loop



MIPS addressing mode summary (Ch. 2, sl. 55)

Examples: lw $t0,32($s3) → $s3 + 32; beq at 80012, offset 2 → 80024; j 20000 → (PC+4)[31:28] : 20000×4.

2020-21 • Q11(c)

CSE 305 • TF page 33 • MARKS 12

Write the equivalent efficient sequence of MIPS instructions for: int testFunc(int a, int b, int c) { int i=0, result=0; do { if (i>b) result += c; i++; } while (i<a); return result; } ($s1 = i, $s2 = result).

SOLUTION

a, b, c arrive in $a0, $a1, $a2. i > b ⇔ b < i.

```
testFunc: add  $s1, $zero, $zero   # i = 0
          add  $s2, $zero, $zero   # result = 0
Loop:     slt  $t0, $a1, $s1       # t0 = 1 if b < i, i.e. i > b
          beq  $t0, $zero, Skip
          add  $s2, $s2, $a2       # result += c
Skip:     addi $s1, $s1, 1         # i++
          slt  $t0, $s1, $a0       # t0 = 1 if i < a
          bne  $t0, $zero, Loop    # do ... while (i < a)
          add  $v0, $s2, $zero     # return result
          jr   $ra
```

Answer. Simulated on four argument sets — equals the C function.

2020-21 • Q11(d)

CSE 305 • TF page 33 • MARKS 5

Provide an illustrative example of how MIPS follows the design principle "Good design demands good compromises".

SOLUTION

MIPS wants every instruction to be 32 bits (regularity), but lw/sw/addi need a constant plus two registers while add needs three registers. Instead of one format with a tiny constant or longer instructions, MIPS adds the **I-format**: the 16-bit constant reuses the rd + shamt + funct bits.

Answer. Decoding becomes slightly harder (the opcode decides the format), but all instructions stay 32 bits and op, rs, rt keep the same positions — a good compromise.

2020–21 • Q12(b)

CSE 305 • TF page 33–34 • MARKS 10

MIPS code L1: bne $t0,$t1,L3; L2: j LX; L3: addi $t0,$t1,1. Opcodes: j = 2, bne = 5, addi = 8; $t0 = 8, $t1 = 9; addresses L1 = 1000, L2 = 1004, L3 = 1008, LX = 131072. Give the format name and field values of each instruction. (Example: add $s1,$s2,$s3 at 0400 is R-format: 0 | 18 | 19 | 17 | 0 | 0.)

SOLUTION

```
1000 L1: bne $t0,$t1,L3   I-format | 5 | 8 | 9 | 1 |      offset = (1008 − 1004)/4 = 1
1004 L2: j LX            J-format | 2 | 32768 |           131072 / 4 = 32768
1008 L3: addi $t0,$t1,1  I-format | 8 | 9 | 8 | 1 |      rs = $t1 = 9, rt = $t0 = 8
```

2020–21 • Q13(b)

CSE 305 • TF page 35 • MARKS 7

How is the address calculation performed in PC-relative addressing?

SOLUTION

Target = PC + 4 + sign-extended 16-bit offset × 4. The PC has already been incremented by 4 when the branch executes; the offset counts words, so it is shifted left by 2; a branch adder adds it.

Answer. Example: beq at 80012 with offset 2 → 80016 + 8 = 80024.

2020–21 • Q13(d)

CSE 305 • TF page 35 • MARKS 5

What are the main differences between x86 architecture and MIPS architecture?

SOLUTION

Feature	MIPS (RISC)	x86 (CISC)
Instruction length	fixed 32 bits, 3 formats	variable 1–15 bytes
Operands	registers only (load/store)	register or memory operand
Operands per ALU instr.	3	2 (destination = a source)
Registers	32 general purpose	8 (IA-32)
Addressing modes	5	many (scaled index, segments)
Conditions	slt / beq / bne	condition codes
Implementation	easy to pipeline, hardwired	microcoded / translated to micro-ops
Code size	larger	denser

2019–20 • Q1(c)

CSE 305 • TF page 36 • MARKS 8

What are the advantages of using Dynamically Linked Libraries (DLL) during a C program execution? What is the role of JIT in Java program execution?

SOLUTION

DLL advantages: no image bloat from statically linking every library; new library versions are picked up automatically; one library copy is shared by many programs; a library is loaded only if called.

JIT: compiles the bytecodes of hot methods into native code while the program runs, so later calls run at native speed (see 2022-23 12(a)).

2019–20 • Q3(a)

CSE 305 • TF page 37–38 • MARKS 15

Write equivalent MIPS for: (i) sum = 0; x = 100; while (x >= −5) { sum = sum + x; x = x − 1; } (ii) if (x > y + 131075) x = 1; (iii) x = A[131075] + 5;

SOLUTION

(i) sum = $s0, x = $s1. (ii) x = $s0, y = $s1. (iii) x = $s0, base of A = $s2.

(i)

```
frag1: add  $s0, $zero, $zero  # sum = 0
       addi $s1, $zero, 100    # x = 100
Loop:  slti $t0, $s1, -5       # t0 = 1 if x < -5
       bne  $t0, $zero, Exit   # leave when !(x >= -5)
       add  $s0, $s0, $s1      # sum = sum + x
       addi $s1, $s1, -1       # x = x - 1
       j    Loop
Exit:  jr   $ra
```

(ii) 131075 = 0x00020003 needs lui/ori.

```
frag2: lui  $t0, 2             # t0 = 0x0002_0000 = 131072
       ori  $t0, $t0, 3        # t0 = 131075 (too big for a 16-bit immediate)
       add  $t0, $s1, $t0      # t0 = y + 131075
       slt  $t1, $t0, $s0      # t1 = 1 if (y + 131075) < x
       beq  $t1, $zero, Skip
       addi $s0, $zero, 1      # x = 1
Skip:  jr   $ra
```

(iii) 131075 × 4 = 524 300 = 0x0008000C does not fit a 16-bit offset.

```
frag3: lui  $t0, 8             # t0 = 0x0008_0000
       ori  $t0, $t0, 12       # t0 = 0x0008_000C = 131075 x 4 = 524300
       add  $t0, $t0, $s2      # t0 = &A[131075]
       lw   $t1, 0($t0)        # t1 = A[131075]
       addi $s0, $t1, 5        # x = A[131075] + 5
       jr   $ra
```

Answer. All three simulated (sum = 5035 for (i); correct branch outcomes for (ii); A[131075] + 5 for (iii)).

2019–20 • Q3(b)

CSE 305 • TF page 38 • MARKS 12

With an appropriate example, briefly describe how pseudoinstructions work. For a critical section where performance matters, is it always a good idea to write the assembly yourself? Justify.

SOLUTION

Pseudoinstructions are accepted by the assembler but not implemented in hardware; each is translated to one or more real instructions, using register **$at**:

```
move $t0,$t1        →  add $t0,$zero,$t1
blt  $t0,$t1,L      →  slt $at,$t0,$t1 ;  bne $at,$zero,L
```

Answer. Not always a good idea. Modern compilers schedule code for modern processors better than most programmers, and more hand-written lines mean more errors and lower productivity (fallacy, Ch. 2 sl. 66). Profile first; hand-code only a small hot spot that the compiler measurably handles badly.

2019–20 • Q3(c)

CSE 305 • TF page 38 • MARKS 8

Why does the MIPS multiplication operation always strictly demand three operands? Why has the number of registers in MIPS been restricted to 32?

SOLUTION

Three operands — simplicity favours regularity: every arithmetic instruction has two sources and one destination, so the decoder, register-file ports and datapath are the same for all (mul rd,rs,rt writes the low 32 bits to rd).

32 registers — smaller is faster: a bigger register file has longer wires and slower access; more registers would also need wider register fields that do not fit three operands in a 32-bit instruction.

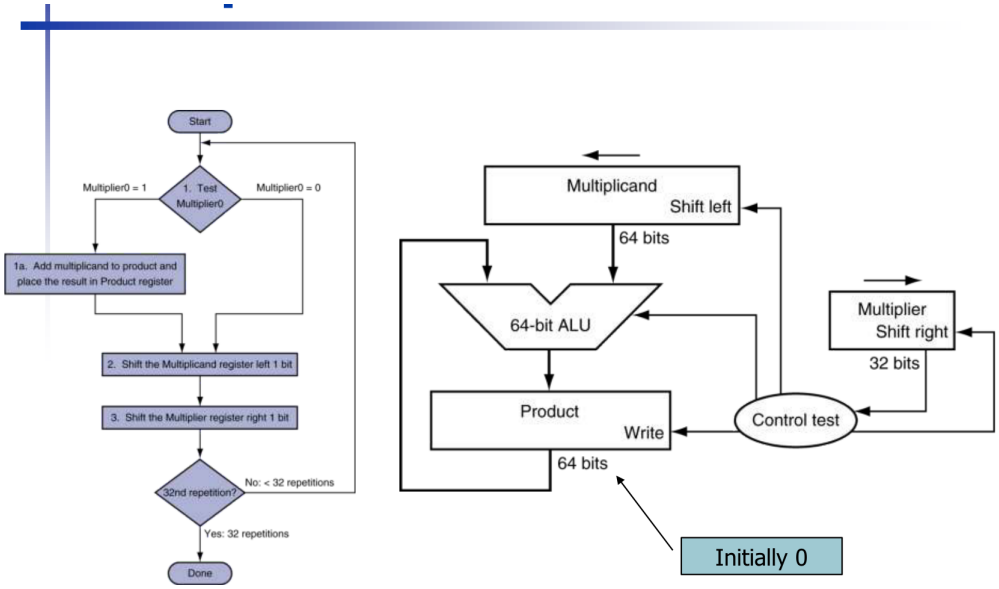
Arithmetic for Computers

Chapter 3 · Lessons L19–L22 · 11 questions

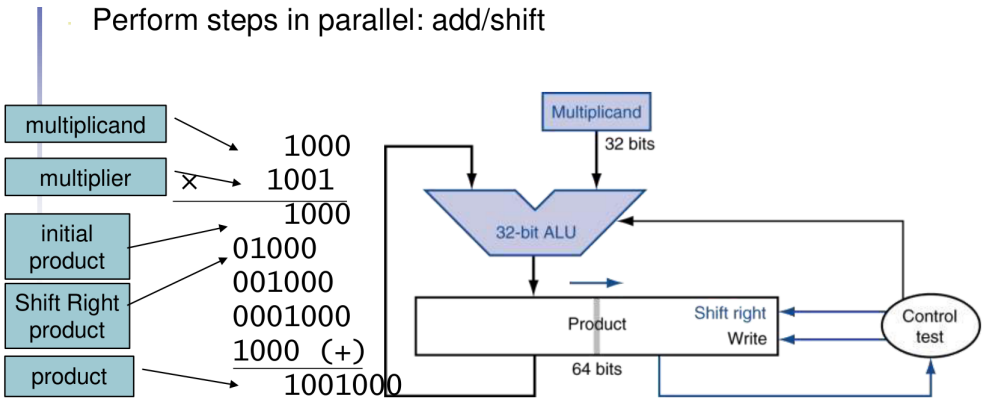
2023-24 • Q7(a)

Draw the integer multiplication hardware following (i) the long-multiplication approach and (ii) the parallel add/shift approach, for 32-bit multiplier and multiplicand. For each approach also simulate the steps for the 4-bit numbers 1000 (multiplicand) and 1011 (multiplier).

SOLUTION



(i) First (long-multiplication) hardware and flowchart: 64-bit multiplicand shifting left, 32-bit multiplier shifting right, 64-bit ALU and product (Ch. 3, sl. 6)



(ii) Optimized hardware: 32-bit multiplicand and ALU, 64-bit product shifting right with the multiplier in its right half (Ch. 3, sl. 7)

Simulation of 1000 × 1011 (8 × 11):

Iter	Step (first version)	Multiplier	Multiplicand	Product
0	Initial values	1011	00001000	00000000
1	1a: Multiplier0 = 1 → Prod = Prod + Mcand	1011	00001000	00001000
	2: Shift left Multiplicand	1011	00010000	00001000
	3: Shift right Multiplier	0101	00010000	00001000
2	1a: Multiplier0 = 1 → Prod = Prod + Mcand	0101	00010000	00011000
	2: Shift left Multiplicand	0101	00100000	00011000
	3: Shift right Multiplier	0010	00100000	00011000
3	1: Multiplier0 = 0 → no add	0010	00100000	00011000
	2: Shift left Multiplicand	0010	01000000	00011000
	3: Shift right Multiplier	0001	01000000	00011000
4	1a: Multiplier0 = 1 → Prod = Prod + Mcand	0001	01000000	01011000

Iter	Step (first version)	Multiplier	Multiplicand	Product
	2: Shift left Multiplicand	0001	10000000	01011000
	3: Shift right Multiplier	0000	10000000	01011000

Iter	Step (refined version)	Multiplicand	Product (left | right = multiplier)
0	Initial: Product = 0000 | multiplier	1000	0000 1011
1	1a: Prod0 = 1 → left half += Mcand	1000	1000 1011
	2: Shift right Product (carry enters MSB)	1000	0100 0101
2	1a: Prod0 = 1 → left half += Mcand	1000	1100 0101
	2: Shift right Product (carry enters MSB)	1000	0110 0010
3	1: Prod0 = 0 → no add	1000	0110 0010
	2: Shift right Product (carry enters MSB)	1000	0011 0001
4	1a: Prod0 = 1 → left half += Mcand	1000	1011 0001
	2: Shift right Product (carry enters MSB)	1000	0101 1000

Answer. Product = 0101 1000 = 88 ✓.

2023-24 • Q7(b)

CSE 209 • TF page 6 • MARKS 15

What is the purpose of a positive bias in IEEE 754? Find the smallest and largest values that can be expressed in (i) single precision (32 bit) and (ii) double precision (64 bit).

SOLUTION

Bias: the exponent is stored as actual + bias (127 / 1023), so the field is an unsigned number: most negative exponent = 00…01, most positive = 11…10. Then two FP numbers of the same sign can be compared with integer comparison hardware, bit pattern by bit pattern.

	Single	Double
Reserved fields	00000000, 11111111	all 0s, all 1s
Smallest normalized	$1.0 \times 2^{-126} \approx 1.18\times10^{-38}$	$1.0 \times 2^{-1022} \approx 2.23\times10^{-308}$
Largest	$(2 - 2^{-23}) \times 2^{127} \approx 3.40\times10^{38}$	$(2 - 2^{-52}) \times 2^{1023} \approx 1.80\times10^{308}$
Smallest denormal	$2^{-149} \approx 1.4\times10^{-45}$	$2^{-1074} \approx 4.9\times10^{-324}$

2022-23 • Q1(a)

CSE 209 • TF page 8 • MARKS 10

Express the steps of floating-point addition using a flow chart. Also write the conditions of overflow, underflow, NaN and infinity during a floating-point addition.

SOLUTION

```
1. Compare exponents; shift the significand of the smaller number right until exponents match
2. Add the significands (with signs)
3. Normalize (shift right/left, increment/decrement exponent);  overflow or underflow? -> exception
4. Round the significand;  still normalized? no -> back to step 3
```

Condition	When (single precision)
Overflow	after normalizing the exponent is > 127 (field would be 255) → ±∞
Underflow	after normalizing the exponent is < −126 → denormal or 0
Infinity	an operand is ±∞ (∞ + x = ∞), or overflow occurred
NaN	an operand is NaN, or (+∞) + (−∞)

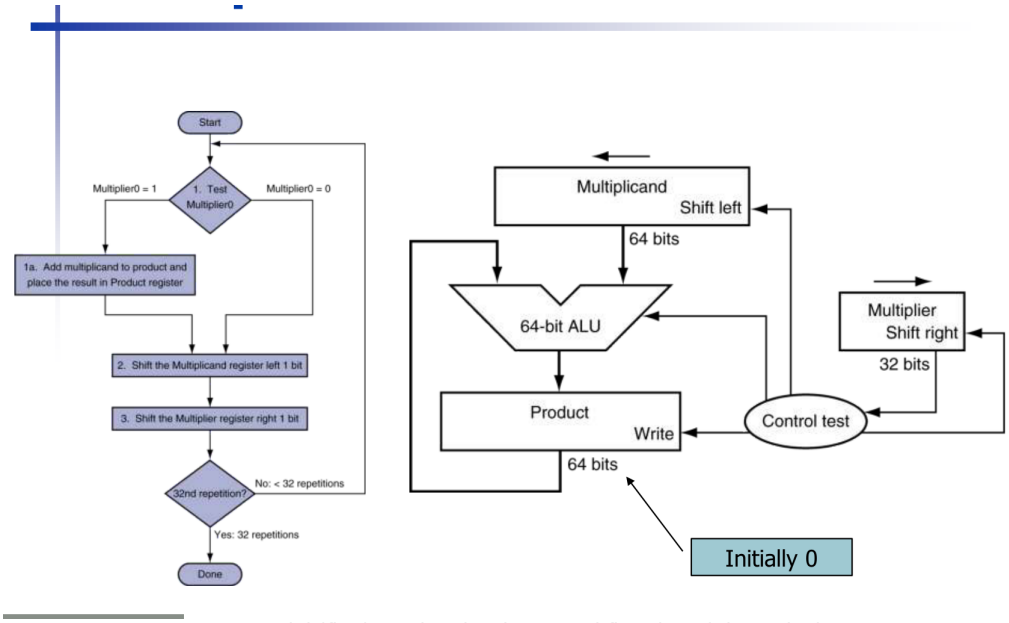
Example (binary): $1.000_2\times2^{-1} + (-1.110_2\times2^{-2}) = 0.5 - 0.4375$. Align: -0.111×2^{-1}; add: 0.001×2^{-1}; normalize: 1.000×2^{-4}; round: unchanged = 0.0625.

2022–23 • Q4

CSE 209 • TF page 10 • MARKS 15

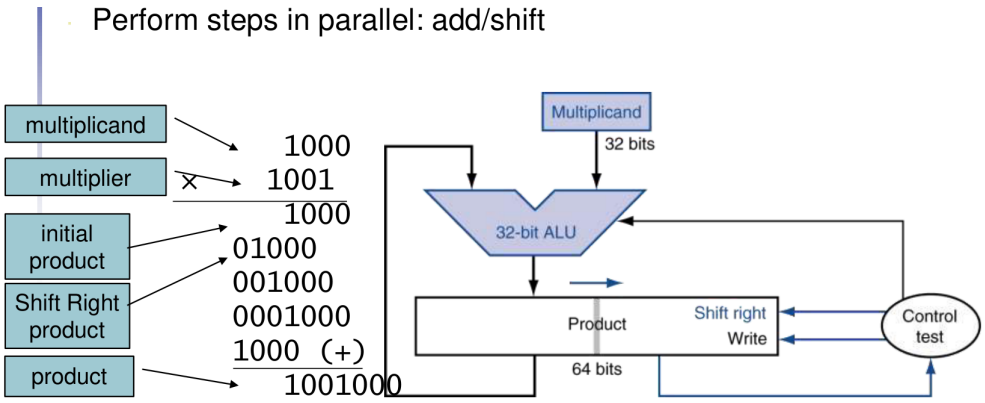
Draw the hardware block diagram and the flowchart of the sequential version of the multiplication algorithm. Also draw the hardware block diagram and flowchart of the refined version.

SOLUTION



Sequential (first) version: hardware and flowchart (Ch. 3, sl. 6)

Flowchart: test Multiplier0; if 1, Product = Product + Multiplicand; shift Multiplicand left 1; shift Multiplier right 1; repeat for 32 repetitions.



Refined version: hardware (Ch. 3, sl. 7)

Flowchart: test Product0; if 1, left half of Product += Multiplicand; shift the Product register right 1 bit; repeat 32 times.

Answer. Improvements: 32-bit multiplicand and ALU instead of 64; no separate multiplier register (it sits in the right half of the product); add and shift happen in the same cycle.

2022–23 • Q8(b)

CSE 305 • TF page 19 • MARKS 8

Consider a 32-bit system where 19 bits are reserved for the fraction, one bit for the sign, and the rest for the exponent. (i) What is the precision range of this system? (ii) What are the smallest and largest absolute values? (iii) What is the smallest denormalized number?

SOLUTION

Exponent bits = $32 - 1 - 19 = 12$, bias = $2^{11} - 1 = 2047$. Normalized exponent range $1 - 2047 = -2046$ … $4094 - 2047 = +2047$.

(i) Precision $2^{-19} \approx 1.9\times10^{-6}$, i.e. $19 \times \log_{10}2 \approx 5.7$ decimal digits.

(ii) Smallest = 1.0×2^{-2046}; largest = $(2 - 2^{-19}) \times 2^{2047}$.

(iii) Smallest denormal = $2^{-19} \times 2^{-2046} = 2^{-2065}$.

2022–23 • Q8(c)

CSE 305 • TF page 19 • MARKS 20

f1 and f2 are normalized IEEE single-precision floats, f1 < f2. (i) If the two 32-bit strings are treated as unsigned integers i1, i2, can we conclude i1 < i2? (ii) If treated as signed integers j1, j2, can we conclude j1 < j2?

SOLUTION

Case	Unsigned i1 < i2?	Signed j1 < j2?
0 ≤ f1 < f2	Yes (sign 0; exponent then fraction order = integer order)	Yes
f1 < 0 ≤ f2	No (i1 has MSB 1 so i1 > i2)	Yes (j1 negative)
f1 < f2 < 0	No (larger magnitude gives larger i)	No (j1 > j2)

Answer. Neither conclusion holds in general. (i) holds only for non-negative numbers; (ii) fails when both are negative. Example: −3.0 = 0xC0400000, −1.0 = 0xBF800000: unsigned 0xC0400000 > 0xBF800000 and signed −1 069 547 520 > −1 082 130 432.

2021–22 • Q6(c)

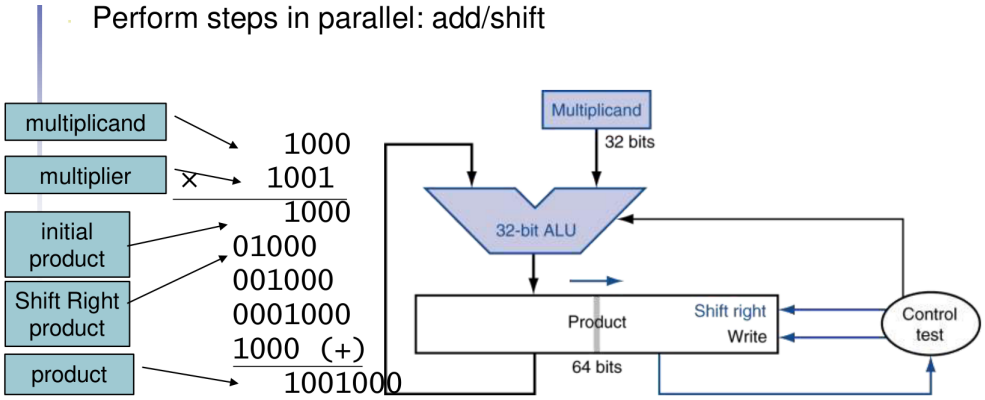
CSE 305 • TF page 24 • MARKS 10

Draw the block diagram of the 32-bit combined multiplication and division hardware that uses a 32-bit ALU, and briefly mention the functions of its components.

SOLUTION

The refined multiplier and the refined divider have the same datapath, so one unit does both:

- **Multiplicand/Divisor register** (32 bits) — holds the second operand.
- **32-bit ALU** — adds (multiply) or subtracts (divide) into the left half of the product/remainder register.
- **Product/Remainder register** (64 bits) — shifts **right** for multiply (product, multiplier in the right half) and **left** for divide (remainder in the left half, quotient bits enter the right half). MIPS names it HI:LO.
- **Control** — tests Product0 (multiply) or the ALU sign (divide), selects add/subtract/shift and counts 32 repetitions.



Refined multiplier datapath (Ch. 3, sl. 7) — the combined unit adds a left-shift path and a subtract control to it

2021–22 • Q8(b)

CSE 305 • TF page 25 • MARKS 14

Consider representing the consecutive positive integers {1, 2, 3, …, n} using IEEE single-precision floating point (8-bit exponent, 23-bit fraction). Find the largest n such that every number in the set can be represented.

SOLUTION

An integer is exact if its binary form fits the 24-bit significand (1 hidden + 23 fraction bits). Every integer up to 2^{24} = 16 777 216 fits ($2^{24} = 1.0 \times 2^{24}$).

$2^{24} + 1 = 1.000\ldots0001 \times 2^{24}$ needs 25 significant bits, so it rounds to 2^{24}.

Answer. n = 2^{24} = 16 777 216 (confirmed numerically).

2020–21 • Q13(a)

CSE 305 • TF page 35 • MARKS 16

Consider a 32-bit system where 24 bits are reserved for the fraction, one bit for the sign and the rest for the exponent. (i) Why is bias used in floating-point representation? (ii) What should be the value of the bias, and why? (iii) What are the smallest and largest absolute values? (iv) What is the smallest denormalized number?

SOLUTION

Exponent bits = 32 − 1 − 24 = 7.

(i) Bias makes the exponent field unsigned, so FP numbers can be compared with integer comparators and a zero exponent field does not mean a negative exponent.

(ii) Bias = $2^6 - 1$ = **63**: it splits the 126 usable exponent values (fields 1…126) into −62 … +63 — nearly symmetric; fields 0 and 127 are reserved.

(iii) Smallest = 1.0×2^{-62}; largest = $(2 - 2^{-24}) \times 2^{63} \approx 1.84\times10^{19}$.

(iv) Smallest denormal = $2^{-24} \times 2^{-62} = \mathbf{2^{-86}}$.

2019–20 • Q4(a)

CSE 305 • TF page 38 • MARKS 15

A 16-bit floating-point system: largest absolute value $1.11\ldots1_2 \times 2^{63}$, smallest $1.00\ldots0_2 \times 2^{-62}$. (i) Show the format. (ii) Represent $-110.11_2 \times 2^1$ with an appropriate bias. (iii) What is the smallest absolute denormalized value?

SOLUTION

Exponents −62 … +63 → 126 normalized values + 2 reserved = 128 = 2^7: **7 exponent bits**, bias 63; fraction = 16 − 1 − 7 = **8 bits**.

```
(i)   | S 1 | Exponent 7 (bias 63) | Fraction 8 |
(ii)  −110.11₂×2¹ = −1.1011₂×2³ → S=1, E = 3+63 = 66 = 1000010, F = 10110000
      1 1000010 10110000 = 0xC2B0   (value −13.5)
(iii) smallest denormal = 2^-8 × 2^(1−63) = 2^-70 ≈ 8.5×10^-22
```

Answer. (i) 1/7/8, (ii) 0xC2B0, (iii) 2^{-70}.

2019–20 • Q4(b)

CSE 305 • TF page 38 • MARKS 12

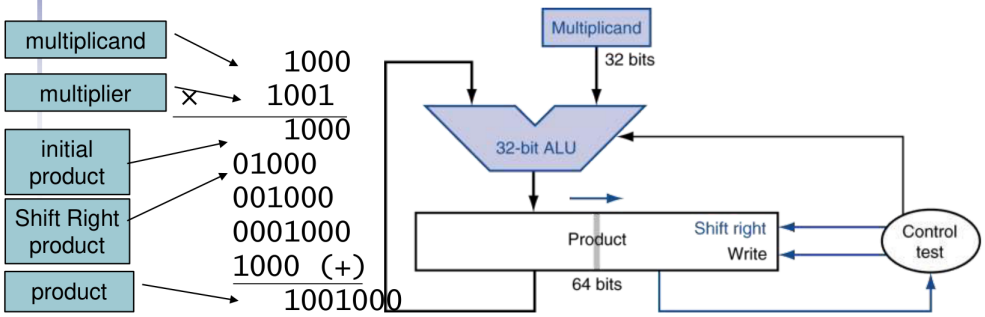
Figure 4b shows the first multiplication hardware (64-bit multiplicand shifting left, 64-bit ALU, 32-bit multiplier shifting right, 64-bit product). Describe why it is inefficient, what changes make it efficient, and draw the efficient hardware.

SOLUTION

Inefficient because: a 64-bit ALU and 64-bit multiplicand register are used although only 32 bits are added per step (half the multiplicand is always 0); a separate multiplier register is needed; and add, shift-left, shift-right are three sequential steps per bit (≈ 3×32 cycles).

Changes: 32-bit multiplicand and 32-bit ALU; shift the **product right** instead of the multiplicand left; keep the multiplier in the unused right half of the 64-bit product register; do the add and the shift in the same cycle → 32 cycles.

- Perform steps in parallel: add/shift



Efficient (refined) multiplication hardware (Ch. 3, sl. 7)

Outside the Short Slides (textbook)

Division, FP code, SPEC, rounding, signed multiplication, subword parallelism — answered from the textbook · 9 questions

2022–23 • Q1(b)

CSE 209 • TF page 8 • MARKS 10

You have a 6-bit ALU and must perform a division where the dividend is 1011 1100 1111 and the divisor is 110101. Determine the result of the division; show the step-by-step calculation with each relevant register at each step.

SOLUTION

Dividend = 3023, divisor = 53. The upper half of the dividend (101111 = 47) is smaller than the divisor, so the 6-bit quotient does not overflow and the refined divider (6-bit ALU, 12-bit remainder register that shifts left) applies:

Iter	Step (refined, 6-bit ALU)	Divisor	Remainder register
0	Initial: Rem = dividend	110101	101111 | 001111
	Shift Rem left 1	110101	1011110 | 011110
1	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	1010010 | 111101
2	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	0111011 | 111011
3	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	0001101 | 110111
4	Left half − Div < 0 → restore; shift Rem left, R0 = 0	110101	0011011 | 101110
5	Left half − Div < 0 → restore; shift Rem left, R0 = 0	110101	0110111 | 011100
6	Left half − Div ≥ 0 → keep; shift Rem left, R0 = 1	110101	0000100 | 111001
	Shift left half right 1 (final fix-up)	110101	000010 111001

Answer. Quotient = 111001_2 = 57, remainder = 000010_2 = 2 (57×53 + 2 = 3023 ✓).

2022–23 • Q8(a)

CSE 305 • TF page 19 • MARKS 7

Write an efficient sequence of MIPS instructions for: float a, b, c; double w, x, y, z; if (a+b > c) x = (y*z)/w; Assume appropriate registers.

SOLUTION

a $f0, b $f1, c $f2 (single); w $f4, x $f6, y $f8, z $f10 (double register pairs).

```
add.s  $f3, $f0, $f1      # a + b
c.le.s $f3, $f2           # (a+b) <= c ?
bc1t   Skip               # true -> condition false, skip
mul.d  $f12, $f8, $f10    # y * z
div.d  $f6, $f12, $f4     # x = (y*z)/w
Skip:
```

2021–22 • Q5(c)

CSE 305 • TF page 24 • MARKS 10

Discuss how subword parallelism can be used to accelerate common operations, such as matrix multiplication or image processing.

SOLUTION

A wide (64/128-bit) adder is partitioned so one instruction works on many short operands at once: 16×8-bit, 8×16-bit, 4×32-bit or 2×64-bit (SIMD — MMX/SSE/AVX, ARM NEON).

Image processing uses 8-bit pixels, so one instruction adds, averages or saturates 16 pixels. **Matrix multiplication** uses packed multiply-add on 4 floats (or 2 doubles) per instruction, cutting the inner-loop instruction count to a quarter or less on the same hardware.

2020–21 • Q10(c)

CSE 305 • TF page 31 • MARKS 5

Briefly describe how to perform a signed multiplication operation.

SOLUTION

Simple method: remember the signs; convert negative operands to positive (2's complement); multiply the magnitudes with the unsigned algorithm (31 iterations); negate the product if the operand signs differ ($S_A \oplus S_B$).

Booth's algorithm works directly on 2's-complement numbers: examine multiplier bits (b_i, b_{i-1}): 10 → subtract the multiplicand, 01 → add it, 00/11 → only shift; shifts are arithmetic (sign-extending).

2020–21 • Q11(a)

CSE 305 • TF page 32 • MARKS 10

SPECBM101: Program 2–6 have execution times 527, 586, 109, 470, 275 s and reference times 10490, 12100, 20720, 7020, 6900 s; Program 1 has reference time 9770 s. If the geometric mean of the benchmark is 28.74 s, what is the execution time of Program 1?

SOLUTION

SPEC ratio = reference time / execution time; the benchmark figure is the geometric mean of the ratios: $(\Pi \text{ ratio}_i)^{1/6}$.

Ratios 2–6: 19.905, 20.649, 190.09, 14.936, 25.091; Π(others) $\approx 2.93\times10^7$ and $28.74^6 \approx 5.64\times10^8$, so $\text{ratio}_1 = 5.64\times10^8 / 2.93\times10^7 = 19.25$.

Answer. $T_1 = 9770 / 19.25 =$ **507.6 s**.

2020–21 • Q11(b)

CSE 305 • TF page 32 • MARKS 8

Figure 11(b) shows a processor unit with a scratchpad memory (MUX for input data, scratchpad with Read/Write and Address, output data feeding registers A and B (Load), ALU with function select, shifter, result back to the MUX). How many micro-operations does an addition of two operands with the result stored require? Which changes and constraints are needed to reduce this to one?

SOLUTION

With one read port feeding registers A and B, the addition takes **3 micro-operations**: (1) A ← M[address of operand 1]; (2) B ← M[address of operand 2]; (3) M[destination] ← A + B (ALU → shifter → MUX → scratchpad write).

Answer. To make it **one** micro-operation the scratchpad must have two read ports and one write port (a register file), the ALU must take both operands directly from the memory outputs (registers A and B removed) and the result must be written back in the same clock cycle (edge-triggered write). Constraint: the two operands must be in different locations addressable at the same time.

2020–21 • Q12(c)

CSE 305 • TF page 34 • MARKS 9

Briefly explain the importance of guard digit, round digit and sticky bit during rounding of an intermediate floating-point number.

SOLUTION

Bits shifted out during alignment or normalization would be lost. IEEE 754 keeps two extra bits, **guard** and **round**, so the result can be rounded as if computed exactly. Example with 3 digits: $2.56\times10^0 + 2.34\times10^2 = 2.3400 + 0.0256 = 2.3656 \rightarrow 2.37$; without them $2.34 + 0.02 = 2.36$ (error in the last digit).

The **sticky bit** is set if any non-zero bit was shifted past the round bit; it lets round-to-nearest-even tell an exact tie (…100…0) from 'more than half', so ties are resolved correctly.

Answer. Together they give results within ½ unit in the last place.

2020–21 • Q13(c)

CSE 305 • TF page 35 • MARKS 7

Write an efficient sequence of MIPS instructions for: float a, b, c; double w, x, y, z; if (a+b>c) x=(y*z)/w;

SOLUTION

Same as 2022-23 CSE 305 8(a):

```
add.s  $f3, $f0, $f1
c.le.s $f3, $f2
bc1t   Skip
mul.d  $f12, $f8, $f10
div.d  $f6, $f12, $f4
Skip:
```

2019–20 • Q4(c)

CSE 305 • TF page 38 • MARKS 8

a, b, c, d and e are of type double with values in $f2, $f4, $f6, $f8 and $f10. Write MIPS for: if (a>b) c = c − (c*e)/d; else c = c + d; using the least number of instructions.

SOLUTION

```
c.le.d $f2, $f4           # a <= b ?
bc1t   Else
mul.d  $f12, $f6, $f10    # c * e
div.d  $f12, $f12, $f8    # (c*e)/d
sub.d  $f6, $f6, $f12     # c = c - (c*e)/d
j      Exit
Else: add.d $f6, $f6, $f8 # c = c + d
Exit:
```